# DocFlow AI - Setup, Deployment and AI Integration README

## 1. Local Development Setup

### 1.1 Clone Project

```
git clone <your-repo-url>
cd docflow_ai
```

### 1.2 Create Virtual Environment

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Linux/Mac:

```
python3 -m venv venv
source venv/bin/activate
```

### 1.3 Install Requirements

```
pip install -r requirements.txt
```

Recommended Django requirements:

```
Django
psycopg2-binary
python-decouple
whitenoise
djangorestframework
django-filter
django-celery-beat
widget-tweaks
django-crispy-forms
crispy-bootstrap5
PyMuPDF
requests
```

## 2. PostgreSQL Setup

Create database:

```
CREATE DATABASE docflow_ai;
```

Example .env:

```
SECRET_KEY=django-insecure-docflow-ai-local-secret-key
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

DATABASE_NAME=docflow_ai
DATABASE_USER=postgres
```

```
DATABASE_PASSWORD=postgres
DATABASE_HOST=localhost
DATABASE_PORT=5432

CSRF_TRUSTED_ORIGINS=http://localhost:8000,http://127.0.0.1:8000

AI_SERVICE_URL=http://localhost:8001
AI_SERVICE_TIMEOUT=15

SESSION_COOKIE_SECURE=False
CSRF_COOKIE_SECURE=False
SECURE_SSL_REDIRECT=False
```

## 3. Django Settings Notes

Important settings:

```
AUTH_USER_MODEL = "accounts.User"
TIME_ZONE = "Asia/Kolkata"
STATIC_ROOT = BASE_DIR / "staticfiles"
MEDIA_ROOT = BASE_DIR / "media"
AI_SERVICE_URL = "http://localhost:8001"
MAX_UPLOAD_SIZE = 10 * 1024 * 1024
```

Use WhiteNoise static storage:

```
STORAGES = {
"default": {
"BACKEND": "django.core.files.storage.FileSystemStorage",
},
"staticfiles": {
"BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",
},
}
```

If manifest errors occur during local development, temporarily use:

```
"BACKEND": "whitenoise.storage.CompressedStaticFilesStorage"
```

## 4. Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Create Super Admin:

```
python manage.py createsuperuser
```

Collect static:

```
python manage.py collectstatic
```

Run server:

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/
```

# 5. FastAPI AI Service Setup

Create separate folder:

```
docflow_ai_service/
■■■ main.py
■■■ requirements.txt
■■■ services/
■■■ __init__.py
■■■ extractor.py
■■■ pdf_reader.py
■■■ risk_detector.py
■■■ summarizer.py
```

Install:

```
cd docflow_ai_service
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

FastAPI requirements:

```
fastapi
uvicorn
pydantic
PyMuPDF
python-multipart
```

Run:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8001
```

Open docs:

```
http://127.0.0.1:8001/docs
```

# 6. AI Endpoints

## Health

```
GET /
GET /health/
```

## Analyze Text

```
POST /ai/analyze-document/
```

Input:

```
{
"text": "Invoice amount Rs. 25000 dated 12/05/2026 approved and signed.",
"filename": "invoice.pdf"
}
```

Output:

```
{
"success": true,
```

```
"filename": "invoice.pdf",
"summary": "This document appears to be related to Vendor Payment...",
"category": "VENDOR_PAYMENT",
"extracted_fields": {
"amount": "25000",
"date": "2026-05-12",
"category": "VENDOR_PAYMENT"
},
"missing_fields": [],
"risk_level": "LOW",
"risk_notes": "No major risk found."
}
```

### Analyze PDF

```
POST /ai/analyze-pdf/
```

Accepts a PDF upload and returns extracted text plus AI fields.

## 7. Django to FastAPI Integration Plan

Create service file:

```
apps/documents/services/ai_client.py
```

Example:

```
import requests
from django.conf import settings


def analyze_document_text(text, filename=None, category=None):
url = f"{settings.AI_SERVICE_URL}/ai/analyze-document/"

payload = {
"text": text,
"filename": filename,
"category": category,
}

response = requests.post(
url,
json=payload,
timeout=settings.AI_SERVICE_TIMEOUT,
)
response.raise_for_status()
return response.json()
```

Then after PDF text extraction:

```
ai_result = analyze_document_text(
attachment.extracted_text,
filename=attachment.original_filename,
)

attachment.ai_summary = ai_result.get("summary", "")
attachment.ai_category = ai_result.get("category", "")
attachment.ai_extracted_data = ai_result.get("extracted_fields", {})
attachment.risk_level = ai_result.get("risk_level", "")
attachment.risk_notes = ai_result.get("risk_notes", "")
attachment.ai_processed = True
attachment.save()
```

## 8. Production Deployment Overview

Recommended production services:

```
Nginx
Gunicorn for Django
Uvicorn for FastAPI
PostgreSQL
Redis
Celery worker
Celery beat
Static files
Media storage
```

## 9. Gunicorn Django Command

```
gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3
```

## 10. FastAPI Production Command

```
uvicorn main:app --host 0.0.0.0 --port 8001 --workers 2
```

## 11. Nginx Routing Concept

```
/ -> Django app on 8000
/static/ -> staticfiles
/media/ -> media
/ai/ -> FastAPI service on 8001
```

Example reverse proxy idea:

```
location / {
proxy_pass http://127.0.0.1:8000;
}

location /ai/ {
proxy_pass http://127.0.0.1:8001/ai/;
}
```

## 12. Static and Media

Collect static:

```
python manage.py collectstatic
```

Media files are stored in:

```
media/
```

In production, protect sensitive uploaded documents. Do not expose confidential files without authentication checks.

## 13. Security Checklist

Before production:

```
DEBUG=False
Strong SECRET_KEY
```

```
Real ALLOWED_HOSTS
Real CSRF_TRUSTED_ORIGINS
HTTPS enabled
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SECURE=True
SECURE_SSL_REDIRECT=True
Database password strong
Media access protected
Regular database backups
Audit logs monitored
Admin URL protected
```

# 14. Troubleshooting

## Conflicting model error

If error says:

```
Conflicting 'organization' models
```

You pasted model code inside admin.py. Keep model only in models.py.

## Static files not updating

Run:

```
python manage.py collectstatic
```

Then browser hard refresh:

```
Ctrl + F5
```

## FastAPI not reachable

Check:

```
http://127.0.0.1:8001/health/
```

Check Django .env:

```
AI_SERVICE_URL=http://localhost:8001
```

## Upload too large

Check:

```
MAX_UPLOAD_SIZE = 10 * 1024 * 1024
DATA_UPLOAD_MAX_MEMORY_SIZE = MAX_UPLOAD_SIZE
FILE_UPLOAD_MAX_MEMORY_SIZE = MAX_UPLOAD_SIZE
```

## URL reverse error

Check app namespace and URL name.

Example template:

```
{% url 'approvals:request_detail' item.pk %}
```

needs:

```
app_name = "approvals"
path("requests/<int:pk>/", views.request_detail, name="request_detail")
```

## 15. Recommended Next Features

```
Notification module
Email notification on approval assignment
Celery AI processing
PDF approval certificate
Advanced reports
Organization user management UI
Reopen/revision flow
Digital signature support
Comment mentions
Dashboard charts
Export audit logs
```